4.1 AI SDK React server components

So far, we've been building conversational UI interfaces with the Next.js app router and have used the Vercel AI SDK to enhance the application features. However, we have not been utilizing the full potential of this framework. For example, we've been developing React components that needed to be shipped to the client and creating dedicated API endpoints to handle text generation and streaming based on user prompts. The generated data needed to be managed securely and sent to the web client to be rendered in the UI.

What if I told you that there is a way to develop React components that work effortlessly on the server side, and we would only need to write code that communicates our intent without manually implementing all the details? This is the essence of RSCs, which are a core feature of Next.js. This section serves as an introduction to this feature, how it works, and how we can use its capabilities to create seamless conversational AI interfaces. By using RSCs, we can focus on the functionality instead of worrying about the nuances of client–server communication, state management, and data fetching.

RSCs offer a new paradigm for building React applications that uses enhanced server-side rendering capabilities. They allow developers to write components that run exclusively on the server, generating HTML that is then sent to the client. This approach offers several advantages, including improved performance, better SEO, and simplified data fetching.

4.1.1 Overview of RSCs

To understand RSCs, we must first grasp how React components are used in a traditional web application. Figure 4.2 shows how the React component tree is typically used.

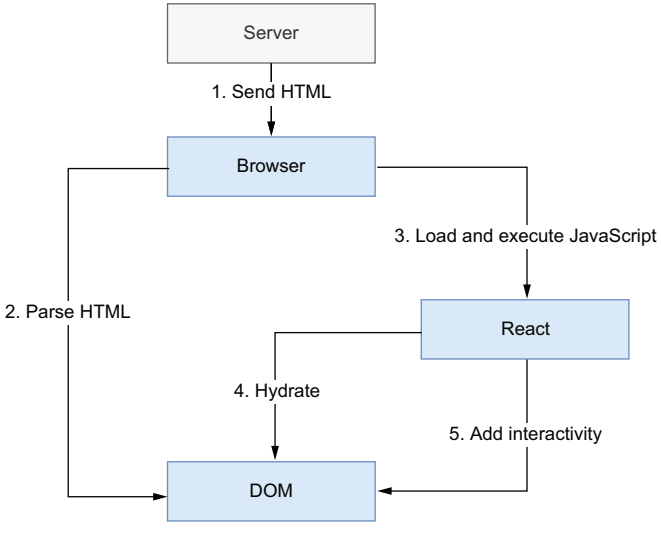


Figure 4.2 Overview of how a traditional React application renders. The React component tree is rendered on the browser. If using server-side rendering, the whole component tree is rendered on the server and then presented to the client. The hydration process then adds interactive elements to the page.